

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

Unidade 4

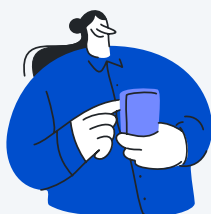
Álgebra Booleana e Lógica Digital

Aula 1

Introdução à Álgebra Booleana

Introdução à álgebra booleana

Introdução à álgebra booleana



Este conteúdo é um vídeo!

Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

Estudante, esta videoaula foi preparada especialmente para você. Nela, você irá aprender conteúdos importantes para a sua formação profissional. Vamos assisti-la?

[Clique aqui](#) para acessar os slides da sua videoaula.

Bons estudos!

Ponto de Partida

Ponto de Partida

Após explorarmos os componentes básicos de um computador, os fundamentos de um sistema computacional e da conversão numérica, agora é hora de elevar o nosso entendimento para o próximo nível. A álgebra booleana é o alicerce sobre o qual repousa a lógica de todo sistema computacional que conhecemos e utilizamos hoje.

A álgebra booleana não é apenas uma área abstrata da matemática; é a linguagem fundamental que descreve o funcionamento interno dos computadores. Ao aprendê-la, você adquire as ferramentas para compreender e projetar a arquitetura lógica que reside no coração de CPUs, memórias e todos os tipos de dispositivos de processamento digital. Cada clique, cada comando, cada programa que você

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

executa é transformado em uma série de expressões booleanas que um computador pode entender e processar. Ao controlar o fluxo de eletricidade por meio de portas lógicas, a álgebra booleana permite a execução de operações complexas a partir de simples combinações de zeros e uns. Portanto, a compreensão e o domínio dessa área nos permitirá entender os aspectos estudados até então em um nível mais profundo e significativo.

Para assimilarmos o conteúdo desta aula de forma prática, considere o projeto de um sistema de segurança com sensores de porta. Em uma casa inteligente, a lógica digital pode ser utilizada para criar um sistema de segurança que sinalize quando alguma porta está aberta ou fechada. Assim como o interruptor controla a passagem de corrente, um sensor de porta pode emitir um sinal lógico indicando o seu estado. Vamos desenvolver um projeto que utiliza um diagrama de lógica digital e conceitos de álgebra booleana para representar o estado das portas de uma casa. Vamos juntos desbloquear o poder dos circuitos lógicos e decifrar a gramática da computação? Bons estudos!

Vamos Começar!

Vamos Começar!

Fundamentos e operações básicas da álgebra booleana

A álgebra booleana, formulada pelo matemático [George Boole no século XIX](#), é um ramo da matemática que se aplica diretamente na arquitetura e na organização de computadores, tornando-se um dos pilares para a compreensão e o desenvolvimento de circuitos digitais e sistemas computacionais. As suas operações e leis são fundamentais para a lógica de programação e processamento de dados digitais (TANGON; DOS SANTOS, 2016).

O cerne da álgebra booleana reside em suas operações básicas: *AND* (conjunção), *OR* (disjunção) e *NOT* (negação), que são as ferramentas elementares para a construção de expressões lógicas. Cada operação possui uma característica definidora que estabelece a base para a manipulação de valores binários, essenciais para o funcionamento dos computadores. Essas operações são visualizadas por meio de tabela verdade, ferramentas indispensáveis para o entendimento do resultado de expressões booleanas

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

complexas. Por exemplo, em uma tabela verdade para a operação *AND*, apenas a combinação de dois valores verdadeiros resultará em verdadeiro, enquanto qualquer inclusão de um valor falso levará a um resultado falso (veremos isto em detalhes mais adiante).

A relevância da álgebra booleana transpõe a teoria matemática e é evidenciada na prática por meio do design de circuitos lógicos. Circuitos simples, como um interruptor, que se baseia no conceito de operação *NOT*, até circuitos mais complexos como somadores e multiplicadores, que combinam múltiplas operações, são projetados e otimizados usando essas operações básicas. Um somador binário, por exemplo, é uma aplicação direta das operações *AND* e *OR*, refletindo a soma de dois dígitos binários e a propagação de um possível *carry-over* (o termo "*carry-over*" no contexto da matemática e da computação refere-se ao valor que é transferido de uma coluna de dígitos para a próxima coluna adjacente em um cálculo aritmético.).

Além disso, as propriedades da álgebra booleana, como comutatividade, associatividade e distributividade, são essenciais para a simplificação de expressões lógicas, tornando possível a criação de circuitos mais eficientes e de menor custo. Por exemplo, a propriedade distributiva pode

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

ser usada para simplificar uma expressão que de outra forma exigiria múltiplas portas lógicas, reduzindo a complexidade física e o consumo de energia do circuito.

Em suma, a álgebra booleana não é apenas uma abstração matemática, mas uma linguagem fundamental que modela a computação. O seu estudo e aplicação são indispensáveis para qualquer pessoa que deseja entender ou trabalhar com tecnologia da informação.

Assim como a álgebra tradicional se apoia no uso de variáveis e operações matemáticas, a álgebra booleana fundamenta-se no emprego de variáveis que expressam estados lógicos binários. Uma variável booleana é caracterizada por poder assumir apenas um de dois valores: 1, que simboliza o estado lógico 'verdadeiro', e 0, que representa o estado 'falso'. Esses estados binários podem ser interpretados de várias formas no mundo real, como verdadeiro ou falso, sim ou não, aberto ou fechado, aceso ou apagado, entre outros.

As operações lógicas elementares da álgebra booleana são: *AND* (conjunção ou E), *OR* (disjunção ou OU) e *NOT* (negação ou NÃO). A tabela a seguir ilustra a simbologia comumente utilizada para representar essas operações lógicas:

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

Operação	Símbolo	Exemplo	Descrição
<i>AND</i>		$A \text{ } B$	O resultado é verdadeiro se A e B forem verdadeiros.
<i>OR</i>		$A \text{ } B$	O resultado é verdadeiro se A ou B (ou ambos) forem verdadeiros.
<i>NOT</i>	$\neg$	$\neg A$	O resultado é o inverso do valor de A; se A é verdadeiro, $\neg A$ é falso, e vice-versa.

Tabela 1 | Simbologia das operações lógicas. Fonte: elaborada pela autora.

Para entendermos melhor, consideremos um sistema de controle de acesso para uma sala de servidores:

- P = Pessoa possui cartão de acesso válido
- D = Pessoa conhece a senha de acesso

Vamos definir as condições para que uma pessoa possa entrar na sala de servidores:

- **P D**: pessoa possui cartão de acesso válido e conhece a senha de acesso. (Nessa condição, o acesso só é

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

permitido se a pessoa tiver ambos: cartão e senha.)

- **P D:** pessoa possui cartão de acesso válido ou conhece a senha de acesso. (Aqui, o acesso é permitido se a pessoa tiver o cartão ou a senha, ou ambos.)
- **$\neg P$:** pessoa não possui cartão de acesso válido. (Essa expressão é verdadeira quando a pessoa não possui o cartão de acesso, independentemente de conhecer a senha.)

Para o sistema de controle, decidimos que o acesso à sala só será permitido sob a condição mais segura, a primeira opção:

- **AND (Produto Lógico):** o sistema de controle verifica o cartão (P) e a senha (D). Se $P = 1$ (verdadeiro) e $D = 1$ (verdadeiro), então o acesso é concedido. Se $P = 0$ ou $D = 0$, o acesso é negado.

Todas as possíveis combinações ficariam mais simples de entender e visualizar utilizando aquilo que chamamos de tabelas verdade, apresentadas a seguir.

Siga em Frente...

Siga em Frente...

Tabelas verdade

As tabelas verdade são uma ferramenta essencial na lógica e na computação, servindo para demonstrar como diferentes valores de verdade (verdadeiro ou falso) são atribuídos a variáveis em expressões lógicas.

Segundo Tanenbaum (2007), as tabelas verdade permitem uma análise sistemática de proposições lógicas, possibilitando entender como as operações lógicas interagem e produzem resultados variados.

Vamos explorar as operações lógicas básicas: *AND*, *OR* e *NOT*, com exemplos práticos:

- **AND (E):** essa operação, também conhecida como conjunção, resulta em verdadeiro apenas se todos os seus operandos forem verdadeiros. Daghlia (2012) explica que na lógica de circuitos, um circuito *AND* produz uma saída verdadeira apenas quando todas as entradas são verdadeiras. Por exemplo, se considerarmos as duas afirmações: "Chove" (A) e "Está

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

frio" (B), a expressão "Chove E Está frio" ($A \text{ AND } B$) só será verdadeira se ambas as afirmações forem verdadeiras simultaneamente.

A	B	$A \text{ AND } B$
V	V	V
V	F	F
F	V	F
F	F	F

Tabela 2 | Tabela *AND*. Fonte: elaborada pela autora.

- **OR (OU):** conhecida como disjunção, essa operação resulta em verdadeiro se pelo menos um dos operandos for verdadeiro. Lourenço *et al.* (2007) descreve que a disjunção reflete a inclusão de múltiplas possibilidades. Em um exemplo prático, se A representa "Está ensolarado" e B representa "Está quente", a expressão "Está ensolarado OU Está quente" ($A \text{ OR } B$) é verdadeira se qualquer uma das condições, ou ambas, forem verdadeiras.

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

A	B	$A \text{ OR } B$
V	V	V
V	F	V
F	V	V
F	F	F

Tabela 3 | Tabela *OR*. Fonte: elaborada pela autora.

- ***NOT(NÃO)***: é uma operação unária, que inverte o valor da variável. Gersting (2017) enfatiza em seus trabalhos a importância da negação para a lógica. Se A é "Estou feliz", *NOTA* representaria "Não estou feliz". A negação transforma uma afirmação verdadeira em falsa e vice-versa. Se "Estou feliz" é verdadeiro, então "Não estou feliz" é falso.

A	<i>NOTA</i>
V	F
F	V

Tabela 4 | Tabela *NOT*. Fonte: elaborada pela autora.

Simplificação de circuitos: portas lógicas

A simplificação de circuitos eletrônicos é um processo que busca reduzir a complexidade de um circuito sem alterar a sua função. Esse processo é fundamental no design de circuitos integrados, em que a eficiência do espaço e a velocidade de processamento são essenciais. Uma técnica comum de simplificação envolve o uso de portas lógicas, que são dispositivos que operam baseados em lógica binária. As portas NAND, XOR, NOR e XNOR são particularmente interessantes, pois podem ser usadas para construir qualquer outra porta lógica, tornando-as instrumentos poderosos na simplificação de circuitos.

- **NAND (Não E):** a porta NAND é uma conjunção negada, ou seja, realiza a função *AND* e, em seguida, inverte o seu resultado. A tabela verdade da porta NAND é uma extensão da *AND*, na qual a saída é falsa somente quando todas as entradas são verdadeiras. A porta NAND é uma porta universal, pois pode ser usada para construir qualquer outra porta lógica.

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

A	B	$(A \text{ B})!$
0	0	1
0	1	1
1	0	1
1	1	0

Tabela 5 | Exemplo de tabela verdade para NAND. Fonte: elaborada pela autora.

- **XOR (Ou Exclusivo):** a porta XOR, ou "ou exclusivo" (), produz uma saída verdadeira quando o número de entradas verdadeiras é ímpar. É frequentemente usada em circuitos de soma e aritmética. A XOR é útil para detectar diferenças entre sinais.

A	B	$A \text{ B}$
0	0	0
0	1	1
1	0	1

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

1	1	0
---	---	---

Tabela 6 | Exemplo de tabela verdade para XOR. Fonte: elaborada pela autora.

- **NOR (Não OU):** a porta NOR é a negação da *OR*. Ela produz uma saída verdadeira apenas se todas as entradas forem falsas. Como Floyd (2007) explica, a porta NOR é também uma porta universal.

A	B	$(A \vee B)!$
0	0	1
0	1	0
1	0	0
1	1	0

Tabela 7 | Exemplo de tabela verdade para NOR. Fonte: elaborada pela autora.

- **XNOR (Não Ou Exclusivo):** a porta XNOR, ou equivalente negado, é o inverso da XOR. Ela produz uma saída

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

verdadeira quando as entradas são iguais. Essa propriedade a torna útil em circuitos de comparação e paridade.

A	B	$(A \oplus B)'$
0	0	1
0	1	0
1	0	0
1	1	1

Tabela 8 | Exemplo de tabela verdade para XNOR. Fonte: elaborada pela autora.

A compreensão e a aplicação dessas portas lógicas são essenciais para estudantes e profissionais que trabalham com eletrônica digital e design de circuitos. A habilidade de simplificar circuitos não apenas economiza recursos mas também pode melhorar o desempenho geral de um sistema eletrônico.

Vamos Exercitar?

Vamos Exercitar?

Vamos agora voltar ao “Ponto de partida” em que apresentamos a ideia de um sistema de segurança com sensores de porta para uma casa inteligente. A nossa tarefa consiste em desenvolver um projeto que utiliza um diagrama de lógica digital e conceitos de álgebra booleana para representar o estado das portas de uma casa.

O primeiro passo é criar um diagrama do circuito com dois sensores de porta que atuam de forma independente. Isso significa que o sistema deve emitir um alerta (ligar uma lâmpada de aviso, por exemplo) se qualquer uma das portas estiver aberta.

Para isso, você usará uma configuração em lógica *OR*, em que a lâmpada acenderá se o Sensor A OU o Sensor B estiver com o sinal em 0 (porta aberta). Nesse caso, o circuito será desenhado de modo que os sinais dos sensores sejam combinados em uma porta *OR*.

Em seguida, crie um diagrama alternativo em que a lâmpada de aviso só acenderá se ambas as portas estiverem abertas

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

ao mesmo tempo. Para essa configuração, você usará uma lógica **AND**, na qual a lâmpada acenderá somente se o Sensor A E o Sensor B estiverem com o sinal em 0.

Para cada um dos diagramas criados, elabore a tabela verdade correspondente, indicando todos os possíveis estados dos sensores e o estado resultante da lâmpada de aviso. Isso demonstrará como a lógica digital reflete o funcionamento do sistema de segurança e como a álgebra booleana pode ser aplicada para entender e projetar sistemas de segurança residencial.

1. Tabela verdade e diagrama - Lógica **OR**:

Sensor A (porta)	Sensor B (porta)	Lâmpada de Aviso (OR)
0	0	1
0	1	1
1	0	1
1	1	0

Tabela 9 | Tabela OR – Sensores. Fonte: elaborada pela autora.

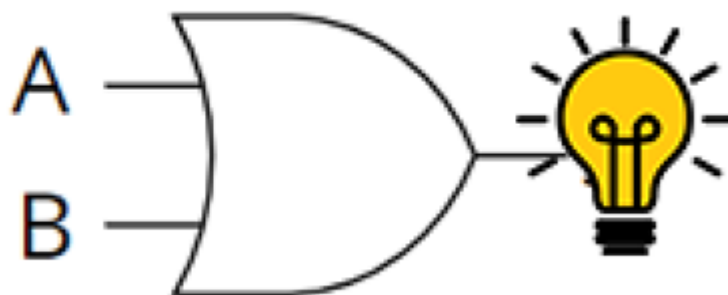


Figura 1 | Diagrama para lógica OR. Fonte: elaborada pela autora.

2. Tabela verdade e diagrama - Lógica *AND*:

Sensor A (porta)	Sensor B (porta)	Lâmpada de Aviso (<i>AND</i>)
0	0	1
0	1	0
1	0	0
1	1	0

Tabela 10 | Tabela AND – Sensores. Fonte: elaborada pela autora.

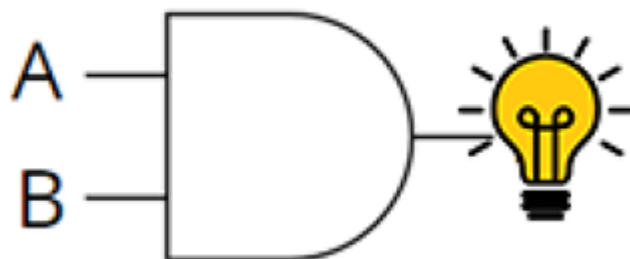


Figura 2 | Diagrama para lógica AND. Fonte: elaborada pela autora.

Saiba mais

Saiba mais

Aprofunde seus conhecimentos sobre os conteúdos desta aula por meio das indicações a seguir:

- Tabelas verdade:

DAGHLIAN, J. Construção da tabela-verdade. *In*: DAGHLIAN, J. [Lógica e álgebra de Boole](#). 4. ed. São Paulo: Atlas, 2012. p. 39-45.

- Simplificação de circuitos:

LOURENÇO, A. C. D. *et al.* Álgebra booleana e circuitos

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

combinacionais. *In*: LOURENÇO, A. C. D. *et al.* **Circuitos digitais – estude e use**. 9. ed. São Paulo: Érica, 2007. p. 46-139.

Referências

Referências

DAGHLIAN, J. **Lógica e álgebra de Boole**. 4. ed. São Paulo: Atlas, 2012.

FLOYD, T. **Sistemas digitais**. 9. ed. Porto Alegre: Bookman, 2007.

GERSTING, J. L. **Fundamentos Matemáticos para a Ciência da Computação**. 7. ed. Rio de Janeiro: LTC, 2017.

LOURENÇO, A. C. D. *et al.* **Circuitos Digitais – Estude e Use**. 9. ed. São Paulo: Érica, 2007.

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

TANENBAUM, A. S. *Organização estruturada de computadores*. 5. ed. São Paulo, SP: Pearson, 2007.

TANGON, L.; DOS SANTOS, R. C. *Arquitetura e Organização de Computadores*. Londrina: Editora e Distribuidora Educacional S.A, 2016.

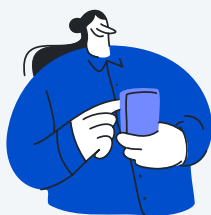
Aula 2

Expressões Lógicas

Expressões lógicas

Expressões lógicas

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES



Este conteúdo é um vídeo!

Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Estudante, esta videoaula foi preparada especialmente para você. Nela, você irá aprender conteúdos importantes para a sua formação profissional. Vamos assisti-la?

[Clique aqui](#) para acessar os slides da sua videoaula.

Bons estudos!

Ponto de Partida

Ponto de Partida

Para prosseguirmos com o nosso estudo sobre as expressões lógicas, é importante que você utilize os

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

conceitos apresentados até aqui, discutimos a lógica booleana, as portas lógicas e as tabelas verdade correspondentes.

A seguir, focaremos na determinação e simplificação de expressões lógicas, resolvendo cada uma de forma lógica, obtendo resultados que podem ser traduzidos em 0 ou 1. Essas simplificações incluem regras de expressões lógicas e teoremas específicos que simplificam a solução dessas expressões, visando diminuir o número de portas lógicas empregadas.

Para colocar em prática o conteúdo da aula, suponhamos que você é um engenheiro eletrônico desafiado a desenvolver um novo sistema de iluminação inteligente para um edifício de escritórios. Esse sistema deve acionar a iluminação (representado pelo sinal de saída 1) de acordo com três condições de sensores ambientais:

- Quando o sensor de luz ambiente (A) detecta que está escuro E o sensor de movimento (B) detecta presença.
- Quando o sensor de luz ambiente (A) detecta que está escuro E o sensor de segurança (C) está desativado (indicando que o prédio não está armado contra intrusos).
- Quando o sensor de movimento (B) detecta presença E o sensor de segurança (C) está desativado.

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

A expressão booleana inicial fornecida para o sistema de iluminação é: $AB + A(B + C) + B(B + C)$. A sua tarefa é simplificar essa expressão booleana para otimizar o uso de portas lógicas na placa de circuito do sistema de iluminação. O objetivo educacional dessa lição é entender e aplicar a determinação e simplificação de expressões lógicas, avançando em nossa competência geral de compreender os princípios da arquitetura e organização de computadores. Enfatizando a simplificação de expressões lógicas por meio de regras e teoremas, nos concentraremos em aprender e praticar métodos de simplificação para alcançar a configuração mais enxuta de portas lógicas, sem perder a equivalência dos resultados. Bons estudos!

Vamos Começar!

Vamos Começar!

Compreensão e manipulação de expressões lógicas

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

A compreensão e a manipulação de expressões lógicas são habilidades fundamentais na área de ciência da computação e engenharia elétrica. Expressões lógicas são usadas para representar operações em circuitos lógicos e em algoritmos computacionais (DAGHLIAN, 2012). Para entender essas expressões, é essencial ter conhecimento sobre a álgebra booleana, um sistema matemático que se baseia em binário, no qual os valores são reduzidos a verdadeiro (1) e falso (0).

Adição booleana

A adição booleana, conhecida como operação **OR**, chamada também de “termo-soma”, é um dos pilares da álgebra booleana, essencial para o entendimento e a construção de circuitos digitais. Em contraste com a adição aritmética convencional, a adição booleana segue regras binárias simples: 0 somado com 0 é 0; 0 somado com 1 é 1; e 1 somado com 1 permanece 1, sem acarretar qualquer "vai um" como aconteceria na aritmética tradicional (TANGON; DOS SANTOS, 2016).

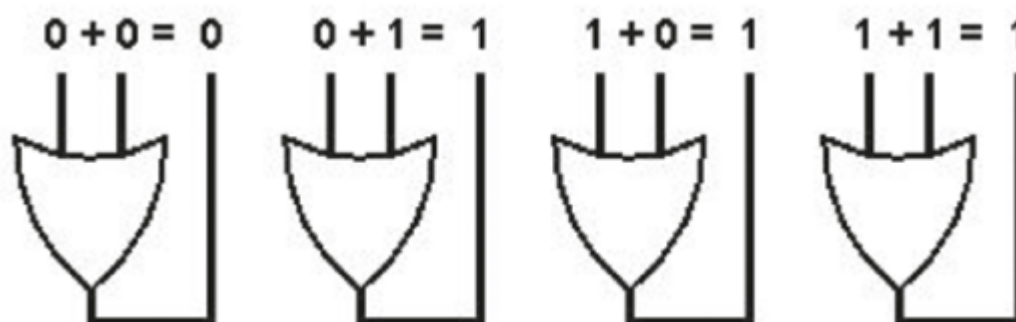
A adição booleana pode ser representada pela tabela verdade a seguir:

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

A (Entrada 1)	B (Entrada 2)	A+B (Saída)
0	0	0
0	1	1
1	0	1
1	1	1

Tabela 1 | Adição booleana. Fonte: elaborada pela autora.

Neste contexto, a operação $A+B$ é verdadeira (1) sempre que **pelo menos um** dos operandos for verdadeiro (1). A adição booleana é não apenas uma operação matemática, mas também uma representação física dentro dos circuitos digitais, em que é realizada por meio de portas lógicas *OR*. Veja um exemplo na figura a seguir:



ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

Figura 1 | Ilustração básica com relação a porta *OR*. Fonte: Tangon e dos Santos (2016, p. 194).

Esta figura apresenta os dados da tabela verdade OR, representado inicialmente pela expressão e, logo após, pelo símbolo lógico.

Exemplos práticos

- **Exemplo 1 - Interruptores de luz:** imagine que A e B representem dois interruptores de luz. Se A ou B (ou ambos) forem acionados, a luz deve acender. Em termos de adição booleana, se A for igual a 1 (interruptor A acionado) ou B for igual a 1 (interruptor B acionado), a luz acende (resultado 1).
- **Exemplo 2 - Lógica de programação:** em programação, a adição booleana é frequentemente aplicada em estruturas condicionais. Por exemplo, se um programa deve executar uma ação quando uma de várias condições for verdadeira, a adição booleana pode ser utilizada para verificar essas condições.

```
if conditionA or conditionB:  
    perform_action()
```

Figura 2 | Exemplo em Python - *OR*. Fonte: elaborada pela autora.

Neste trecho de código, *perform_action()* é executado se *conditionA* for verdadeira (1), *conditionB* for verdadeira (1), ou ambas forem verdadeiras.

Multiplicação booleana

A multiplicação booleana é um conceito fundamental, mais conhecida como a operação **AND**, ou ainda “termo-produto” (TANGON e DOS SANTOS, 2016). A multiplicação booleana segue uma lógica binária direta: 1 multiplicado por 1 é 1; **qualquer outra multiplicação resulta em 0.**

Esta operação é representada por uma tabela verdade que define claramente as regras para a multiplicação de dois valores booleanos.

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

A (Entrada 1)	B (Entrada 2)	A · B (Saída)
0	0	0
0	1	0
1	0	0
1	1	1

Tabela 2 | Multiplicação booleana. Fonte: elaborada pela autora.

Na multiplicação booleana, o resultado é verdadeiro (1) apenas quando ambos os operandos são verdadeiros (1). Essa propriedade é fundamental no projeto de circuitos digitais, em que a operação é implementada por meio de portas lógicas *AND* (FLOYD, 2007). Veja, como exemplo, a figura a seguir:

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

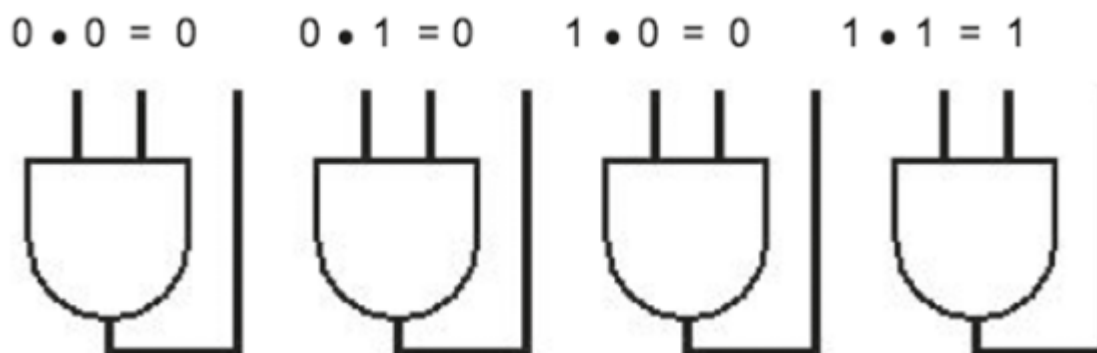


Figura 3 | Ilustração básica com relação a porta AND. Fonte: Tangon e dos Santos (2016, p. 195).

Exemplos práticos:

- **Exemplo 1 - Sistema de controle de acesso:** imagine um sistema de controle de acesso que requer dois fatores de autenticação: um cartão (A) e uma senha (B). Ambos os fatores devem ser validados para conceder acesso. Em termos booleanos, $A \cdot B$ deve ser igual a 1 para que a porta se abra, o que só ocorre se $A = 1$ (cartão presente) e $B = 1$ (senha correta).
- **Exemplo 2 - Programação e lógica:** a multiplicação booleana também é fundamental na lógica de programação. Quando uma ação deve ser tomada apenas se várias condições forem simultaneamente satisfeitas, a multiplicação booleana é utilizada.

```
if conditionA and conditionB:  
    perform_action()
```

Figura 4 | Exemplo em Python - AND. Fonte: elaborada pela autora.

Siga em Frente...

Siga em Frente...

Leis e regras da álgebra booleana

A álgebra booleana, desenvolvida por George Boole no século XIX, é um pilar da matemática discreta e da ciência da computação. Ela se baseia em valores binários - 0 e 1 - para representar o verdadeiro e o falso, respectivamente, e se utiliza de um conjunto de leis e regras para a manipulação desses valores, essencial para o design e a análise de sistemas eletrônicos e lógicos (GERSTING, 2017).

Leis comutativas

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

As leis comutativas se aplicam tanto à adição quanto à multiplicação booleana, afirmando que a ordem dos operandos não afeta o resultado.

- Adição: $A + B = B + A$
- Multiplicação: $A \cdot B = B \cdot A$

Exemplo: se A representa um sensor de porta e B um sensor de janela, a lei comutativa da adição sugere que o alarme será ativado (1) se qualquer sensor for ativado, independente da ordem.

Leis associativas

As leis associativas permitem o reagrupamento dos operandos sem alterar o resultado.

- Adição: $(A + B) + C = A + (B + C)$
- Multiplicação: $(A \cdot B) \cdot C = A \cdot (B \cdot C)$

Exemplo: se temos três sensores A, B, e C, a lei associativa da multiplicação indica que o sistema acionará um mecanismo de segurança se todos os sensores forem ativados

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

simultaneamente, independentemente de como eles são agrupados.

Lei distributiva

A lei distributiva conecta adição e multiplicação, permitindo a distribuição de um operando sobre a adição ou a subtração dos outros dois.

- Distributiva: $A \cdot (B + C) = A \cdot B + A \cdot C$

Exemplo: em um circuito, se A representa um interruptor principal e B e C representam dois circuitos secundários, a lei distributiva pode ser usada para projetar um sistema em que A ativa B e C individualmente.

Doze regras básicas da álgebra de Boole

1. Elemento de identidade:

- Adição: $A + 0 = A$
- Multiplicação: $A \cdot 1 = A$

2. Elemento de dominação:

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

- Adição: $A + 1 = 1$
- Multiplicação: $A \cdot 0 = 0$

3. Idempotência:

- Adição: $A + A = A$
- Multiplicação: $A \cdot A = A$

4. Complementos:

- $A + A' = 1$
- $A \cdot A' = 0$

5. Dupla negativa (Involução): $(A')' = A$

6. Comutativa:

- Adição: $A + B = B + A$
- Multiplicação: $A \cdot B = B \cdot A$

7. Associativa:

- Adição: $(A + B) + C = A + (B + C)$

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

- Multiplicação: $(A \cdot B) \cdot C = A \cdot (B \cdot C)$

8. Distributiva:

- $A \cdot (B + C) = A \cdot B + A \cdot C$

9. Absorção:

- $A + A \cdot B = A$
- $A \cdot (A + B) = A$

10. De Morgan:

- $(A + B)' = A' \cdot B'$
- $(A \cdot B)' = A' + B'$

11. Lei do Cancelamento:

- $A + AB = A$
- $A(A + B) = A$

12. Lei da Consistência:

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

- $(A + B) (A + B') = A$

Exemplo para as regras básicas

Em uma rede de computadores, A pode representar a conexão com a rede, B um *firewall* e C um sistema de backup. A Lei da Absorção sugere que se a rede estiver ativa (A), então a presença (ou ausência) do *firewall* ou do backup ($A \cdot B$ ou $A \cdot C$) não altera o estado da rede.

Teoremas de De Morgan

Os Teoremas de De Morgan são fundamentais na álgebra booleana e no design de circuitos lógicos. Eles fornecem uma maneira sistemática de transformar expressões lógicas complexas em suas formas complementares, o que é crucial para a simplificação de circuitos e a realização de análises lógicas (TANENBAUM, 2007). Existem dois Teoremas de De Morgan:

1. O complemento de uma conjunção (*AND*) é equivalente à disjunção (*OR*) dos complementos individuais.
2. O complemento de uma disjunção (*OR*) é equivalente à conjunção (*AND*) dos complementos individuais.

Matematicamente, eles podem ser expressos como:

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

$$\begin{aligned} 1. \overline{(A \cdot B)} &= \overline{A} + \overline{B} \\ 2. \overline{(A + B)} &= \overline{A} \cdot \overline{B} \end{aligned}$$

Aplicações dos teoremas

Os teoremas são aplicados na simplificação de circuitos, na conversão entre lógica positiva e negativa, e na implementação de funções lógicas com um número limitado de tipos de portas lógicas.

- **Exemplo 1 - Simplificação de circuitos:** suponha que você tenha um circuito com uma porta *AND* seguida por uma porta *NOT*. De acordo com o primeiro teorema de De Morgan, isso pode ser substituído por duas portas *NOT* seguidas por uma porta *OR*, o que pode ser mais conveniente dependendo do design do circuito.
- **Exemplo 2 - Lógica positiva para negativa:** considere um sistema que utiliza lógica positiva, em que '1' representa verdadeiro. Se você precisar converter esse sistema para usar lógica negativa, em que '0' representa verdadeiro, os Teoremas de De Morgan podem ser usados para reescrever as expressões lógicas apropriadamente.

Exemplos práticos

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

- Exemplo prático de um circuito:

Imagine um sistema de segurança que aciona um alarme se todas as portas A, B e C estiverem fechadas simultaneamente. A lógica seria $A \cdot B \cdot C$. Se quisermos projetar um sistema que acione o alarme quando qualquer porta estiver aberta, podemos negar a expressão original e aplicar os Teoremas de De Morgan, resultando em $A' + B' + C'$.

- Exemplo de programação:

Em um contexto de programação, considere uma função que deve executar uma ação se determinadas condições não forem atendidas. Se as condições forem `'user_logged_in'` e `'user_has_permission'`, a lógica para executar uma ação se o usuário não estiver logado ou não tiver permissão seria:

```
if not (user_logged_in and user_has_permission):  
    take_action()
```

Figura 5 | Exemplo em Python de condição de negação.

Fonte: elaborada pela autora.

Vamos ver um exemplo de utilização agora. Iremos reduzir a expressão $(AB + C)$ usando o teorema 1:

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

$\overline{(AB + C)} = \overline{(AB)} \cdot \overline{C}$ Aqui, consideramos AB como o "A" e C como o "B". Essa expressão ainda pode ser mais simplificada (pois temos um produto AB que é invertido). Podemos aplicar agora o teorema (2):

$$\overline{(AB)} \cdot \overline{C} = (\overline{A} + \overline{B}) \cdot \overline{C}$$

Agora, finalmente, teremos:

$$(\overline{A} + \overline{B}) \cdot \overline{C} = \overline{A}\overline{C} + \overline{B}\overline{C}$$

Nesse resultado, encontramos apenas variáveis simples e sinais de inversão.

Concluimos tendo demonstrado que os Teoremas de De Morgan são ferramentas poderosas na manipulação e simplificação de expressões booleanas. Eles permitem que os engenheiros e cientistas da computação convertam e simplifiquem circuitos e algoritmos, tornando-os mais eficientes e mais fáceis de entender e implementar.

Vamos Exercitar?

Vamos Exercitar?

No “Ponto de partida”, propusemos a seguinte atividade:

Você, sendo um engenheiro eletrônico, foi encarregado de desenvolver um novo sistema de iluminação inteligente para um edifício de escritórios. Esse sistema deve acionar a iluminação (representado pelo sinal de saída 1) de acordo com as três condições de sensores ambientais:

- Quando o sensor de luz ambiente (A) detecta que está escuro E o sensor de movimento (B) detecta presença.
- Quando o sensor de luz ambiente (A) detecta que está escuro E o sensor de segurança (C) está desativado (indicando que o prédio não está armado contra intrusos).
- Quando o sensor de movimento (B) detecta presença E o sensor de segurança (C) está desativado.

A expressão booleana inicial fornecida para o sistema de iluminação é $AB + A(B + C) + B(B + C)$. A sua tarefa era simplificar essa expressão booleana para otimizar o uso de portas lógicas na placa de circuito do sistema de iluminação.

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

Para solucionar o problema apresentado, primeiro, vamos aplicar as leis da álgebra booleana para simplificar a expressão dada:

1. Aplicar a lei distributiva no segundo e terceiro termo:

$$AB + AB + AC + BB + BC =$$

2. Aplicar a regra 5 ($AB + AB = AB$) nos dois primeiros termos da expressão:

$$AB + AC + BB + BC =$$

3. Agora, aplique a regra 7 ($BB = B$) no terceiro termo:

$$AB + AC + B + BC =$$

4. Aplicar a regra 10 ($B + BC = B$) no terceiro e quarto termo:

$$AB + AC + B$$

5. Para finalizar, aplicamos a regra 10 ($AB + B = B$) novamente, agora no primeiro termo e no último termo:

$$B + AC$$

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

Desse modo, não conseguimos mais simplificar a expressão, chegando em sua total simplificação. A expressão simplificada $B + AC$ representa o menor número de portas possíveis para a expressão lógica $B + A(B + C) + B(B + C)$.

Saiba mais

Saiba mais

Que tal aprender mais sobre os conteúdos desta aula?
Confira as indicações a seguir:

- Compreensão e manipulação de expressões lógicas

ALVES, W. P. Introdução à Lógica. *In*: ALVES, W. P. [Linguagem e Lógica de Programação](#). 1. ed. São Paulo: Érica, 2014. p. 10-15.

- Leis e regras da álgebra booleana

DAGHLIAN, J. Introdução à álgebra de Boole. *In*: DAGHLIAN, J. [Lógica e álgebra de Boole](#). 4. ed. São Paulo: Atlas, 2012. p.

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

97-116.

Referências

Referências

DAGHLIAN, J. **Lógica e álgebra de Boole**. 4. ed. São Paulo: Atlas, 2012.

FLOYD, T. **Sistemas digitais**. 9. ed. Porto Alegre: Bookman, 2007.

GERSTING, J. L. **Fundamentos Matemáticos para a Ciência da Computação**. 7. ed. Rio de Janeiro: LTC, 2017.

TANENBAUM, A. S. **Organização estruturada de computadores**. 5. ed. São Paulo, SP: Pearson, 2007.

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

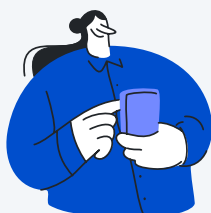
TANGON, L.; DOS SANTOS, R. C. **Arquitetura e Organização de Computadores**. Londrina: Editora e Distribuidora Educacional S.A, 2016.

Aula 3

Portas Lógicas

Portas lógicas

Portas lógicas



Este conteúdo é um vídeo!

Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

Estudante, esta videoaula foi preparada especialmente para você. Nela, você irá aprender conteúdos importantes para a sua formação profissional. Vamos assisti-la?

[Clique aqui](#) para acessar os slides da sua videoaula.

Bons estudos!

Ponto de Partida

Ponto de Partida

Olá, estudante! Adentramos em um território fundamental da eletrônica digital: o mundo das portas lógicas. Estas não são simplesmente componentes eletrônicos; são os alicerces da lógica que impulsiona as inovações tecnológicas, desde os simples dispositivos domésticos até complexos sistemas computacionais.

Agora, é hora de aplicar os seus conhecimentos teóricos em um contexto prático que simula um desafio real enfrentado pelos engenheiros de sistemas. Na seção anterior, foram introduzidas as expressões lógicas e os métodos pelos quais elas podem ser simplificadas. É imprescindível destacar a importância das tabelas verdade, ferramenta já explorada, para aprofundarmos o nosso entendimento sobre as portas

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

lógicas. Essas tabelas são fundamentais para visualizar e compreender o funcionamento das portas lógicas e, por extensão, para o desenvolvimento eficaz de circuitos lógicos. Para absorver o conteúdo de forma prática, imagine que você está prestes a embarcar na criação de algo tangível e funcional: o diagrama de um circuito impresso. Esse circuito será a base de um projeto de uma porta automática, cujo movimento é governado pela lógica digital que você vai projetar.

- A saída '1' sinaliza a abertura da porta.
- As entradas são definidas como:

$p = 1$ quando uma pessoa é detectada.

$q = 1$ quando a chave para abertura é ativada.

$z = 1$ quando a chave para fechamento é ativada.

O desafio é desenvolver um diagrama em que a porta se abre se, e somente se, a entrada ($q = 1$ e $z = 0$) ou ($q = 0$ e $p = 1$ e $z = 0$). Esse cenário prático exige que você utilize portas lógicas adequadas para criar um circuito que atenda a essas condições específicas (TANGON; DOS SANTOS, 2016).

Ao final desta aula, você terá o conhecimento e a experiência

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

de ter projetado um diagrama de circuito impresso funcional, um passo inicial e essencial na jornada de engenharia eletrônica e design de sistemas. Bons estudos!

Vamos Começar!

Vamos Começar!

Fundamentos de portas lógicas

As portas lógicas são a base de todos os sistemas digitais e eletrônicos modernos, funcionando como os neurônios no cérebro de um computador. Elas realizam operações fundamentais baseadas em álgebra booleana, que processam entradas binárias para produzir uma saída específica. Floyd (2007) descreve portas lógicas como o alicerce que permite que dispositivos digitais executem cálculos e tomem decisões lógicas.

Ao abordarmos o exemplo de circuitos integrados complexos, estamos nos referindo a sistemas digitais completos, prontos para serem empregados em diversas aplicações. Os “microcontroladores” e processadores são

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

exemplares clássicos desses circuitos integrados, representando o ápice da complexidade e da funcionalidade na eletrônica digital. Na eletrônica digital, a simplicidade do processamento decorre do uso exclusivo de dois dígitos: 0 e 1, conhecidos como um bit. Esses dígitos correspondem, respectivamente, a dois distintos níveis de tensão: "0" representando 0 volts e "1" representando 5 volts. Essa dualidade facilita o processamento e a representação de informações (TANGON; DOS SANTOS, 2016).

As representações gráficas das portas lógicas, fundamentais na eletrônica digital, são habitualmente expressas por meio das entradas A e B, e da saída S. Essa simbologia, referida como blocos lógicos, estabelece a interconexão entre as entradas e a saída, em que as entradas e saídas são restritas aos valores binários de 0 ou 1, conforme descrito por Tocci e Widmer (2011). A relação entre as entradas lógicas e os resultados é sempre determinada por uma tabela verdade, que será detalhadamente explorada na discussão subsequente sobre a simbologia específica de cada porta lógica.

Siga em Frente...

Siga em Frente...

Tipos e representação de portas lógicas

Existem várias portas lógicas básicas, cada uma realizando uma função booleana distinta. Cada tipo de porta lógica possui um símbolo gráfico distinto que a identifica em diagramas de circuitos. Esses símbolos são uma linguagem universal em eletrônica digital, permitindo que os engenheiros e designers compartilhem e interpretem esquemas de circuitos com clareza e precisão. As funções das portas lógicas também podem ser descritas por meio de expressões matemáticas que utilizam variáveis e operadores booleanos. Essas expressões são equivalentes à operação que a porta executa sobre as entradas para produzir uma saída (STALLINGS, 2017).

Vejamos cada tipo em detalhe a seguir.

- Inversor (NOT ou Negação)

O inversor, comumente chamado de porta NOT, é uma porta lógica que implementa a negação: se a entrada é verdadeira

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

(1), a saída será falsa (0), e vice-versa.

Tabela verdade do inversor:

A (Entrada)	NOTA (Saída)
0	1
1	0

Tabela 1 | Tabela verdade - Inversor. Fonte: elaborada pela autora.

Expressão da função:

$$S = \overline{A}$$

O travessão acima da variável A é comumente utilizado como representação da sua negação.

Símbolo:

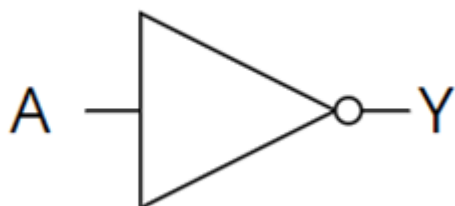


Figura 1 | Inversor. Fonte: elaborada pela autora.

- Porta *OR*

A porta *OR* produz uma saída verdadeira (1) se pelo menos uma das entradas for verdadeira (1). Se todas as entradas forem falsas (0), a saída será falsa (0).

Tabela verdade da porta OR:

A (Entrada)	B (Entrada)	A OR B (Saída)
0	0	0
0	1	1
1	0	1
1	1	1

Tabela 2 | Tabela verdade - *OR*. Fonte: elaborada pela autora.

Expressão da função:

$$S = A + B$$

Símbolo:

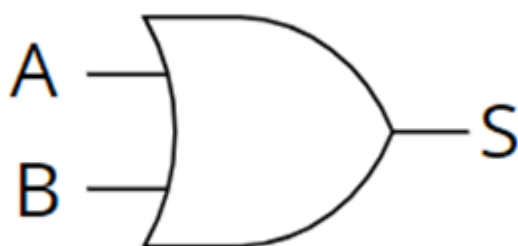


Figura 2 | Porta *OR*. Fonte: elaborada pela autora.

- Porta *AND*

A porta *AND* produz uma saída verdadeira (1) somente se todas as entradas forem verdadeiras (1). Se qualquer entrada for falsa (0), a saída será falsa (0).

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

Tabela verdade da porta AND:

A (Entrada)	B (Entrada)	A AND B (Saída)
0	0	0
0	1	0
1	0	0
1	1	1

Tabela 3 | Tabela verdade - AND. Fonte: elaborada pela autora.

Expressão da função:

$$S = A \cdot B$$

Símbolo:

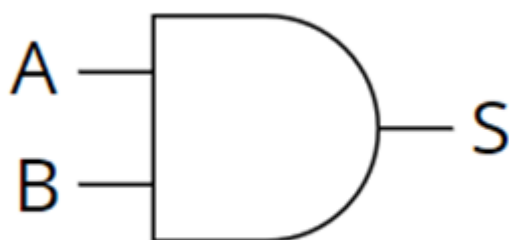


Figura 3 | Porta AND. Fonte: elaborada pela autora.

- Porta NAND

A porta NAND é o inverso da porta AND. Ela produz uma saída falsa (0) somente se todas as entradas forem verdadeiras (1).

Tabela verdade da porta NAND:

A (Entrada)	B (Entrada)	A NAND B (Saída)
0	0	1
0	1	1
1	0	1

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

1	1	0
---	---	---

Tabela 4 | Tabela verdade - NAND. Fonte: elaborada pela autora.

Expressão da função:

$$S = \overline{A \cdot B}$$

Símbolo:

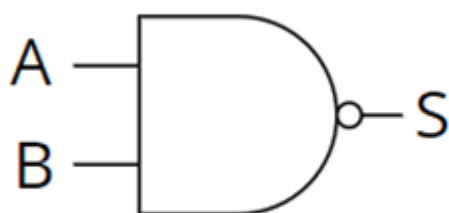


Figura 4 | Porta NAND. Fonte: elaborada pela autora.

- Porta NOR

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

A porta NOR é o inverso da porta *OR*. Ela produz uma saída verdadeira (1) somente se todas as entradas forem falsas (0).

Tabela verdade da porta NOR:

A (Entrada)	B (Entrada)	A NOR B (Saída)
0	0	1
0	1	0
1	0	0
1	1	0

Tabela 5 | Tabela verdade - NOR. Fonte: elaborada pela autora.

Expressão da função:

$$S = \overline{A + B}$$

Símbolo:

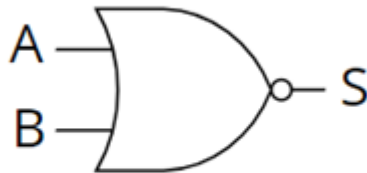


Figura 5 | Porta NOR. Fonte: elaborada pela autora.

- Porta XOR

A porta XOR, ou "*exclusive OR*", produz uma saída verdadeira (1) se um número ímpar de entradas for verdadeiro (1). Se um número par de entradas for verdadeiro, ou todas forem falsas, a saída será falsa (0).

Tabela verdade da porta XOR:

A (Entrada)	B (Entrada)	A XOR B (Saída)
0	0	0
0	1	1
1	0	1

1	1	0
---	---	---

Tabela 6 | Tabela verdade - XOR. Fonte: elaborada pela autora.

Expressão da função:

$$S = A \oplus B$$

Símbolo:

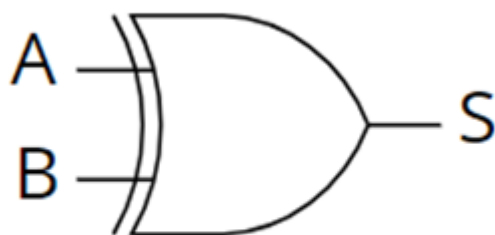


Figura 6 | Porta XOR. Fonte: elaborada pela autora.

- Porta XNOR

A porta XNOR, ou "*exclusive NOR*", é o inverso da porta XOR. Ela produz uma saída verdadeira (1) se um número par de

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

entradas for verdadeiro (1) ou se todas as entradas forem falsas (0).

Tabela verdade da porta XNOR:

A (Entrada)	B (Entrada)	A XNOR B (Saída)
0	0	1
0	1	0
1	0	0
1	1	1

Tabela 7 | Tabela verdade - XNOR. Fonte: elaborada pela autora.

Expressão da função:

$$S = \overline{A \oplus B} \text{ ou } S = A \odot B$$

Símbolo:

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

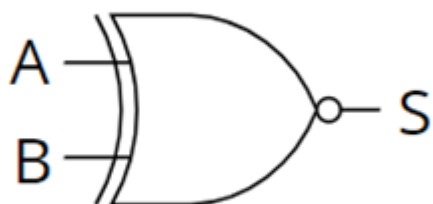


Figura 7 | Porta XNOR. Fonte: elaborada pela autora.

Cada tipo de porta lógica tem um papel único no design de circuitos digitais, e a compreensão de suas funções é essencial para a engenharia de sistemas eletrônicos e computacionais. As tabelas verdade e expressões funcionais são ferramentas fundamentais para a análise e implementação de lógica digital em diversas aplicações práticas.

Os fundamentos das portas lógicas são essenciais para qualquer pessoa que deseje entender ou trabalhar com tecnologia digital. Conforme expresso por Tanenbaum (2007), um sólido entendimento das portas lógicas e de como elas interagem é fundamental para o sucesso no campo da eletrônica digital. Ao dominar esses conceitos, abrem-se inúmeras possibilidades para a criação, inovação e resolução de problemas no vasto mundo da tecnologia.

Vamos Exercitar?

Vamos Exercitar?

No “Ponto de partida”, propusemos a elaboração de um circuito base de um projeto de uma porta automática:

- A saída '1' sinaliza a abertura da porta.
- As entradas são definidas como:

$p = 1$ quando uma pessoa é detectada.

$q = 1$ quando a chave para abertura é ativada.

$z = 1$ quando a chave para fechamento é ativada.

O desafio era desenvolver um diagrama em que a porta se abre se, e somente se, a entrada ($q = 1$ e $z = 0$) ou ($q = 0$ e $p = 1$ e $z = 0$).

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

Tendo adquirido as informações anteriores necessárias para a elaboração do diagrama, é necessário agora traduzir esses dados em uma expressão lógica correspondente, que pode ser articulada da seguinte maneira:

$$S = q\bar{z} + \bar{q}p\bar{z}$$

Com base na expressão fornecida, é possível discernir que o diagrama a ser desenvolvido incorporará portas lógicas *AND*, *OR*, além de inversores (*NOT*). De acordo com a expressão lógica estabelecida, o diagrama será estruturado conforme a representação gráfica que se segue.

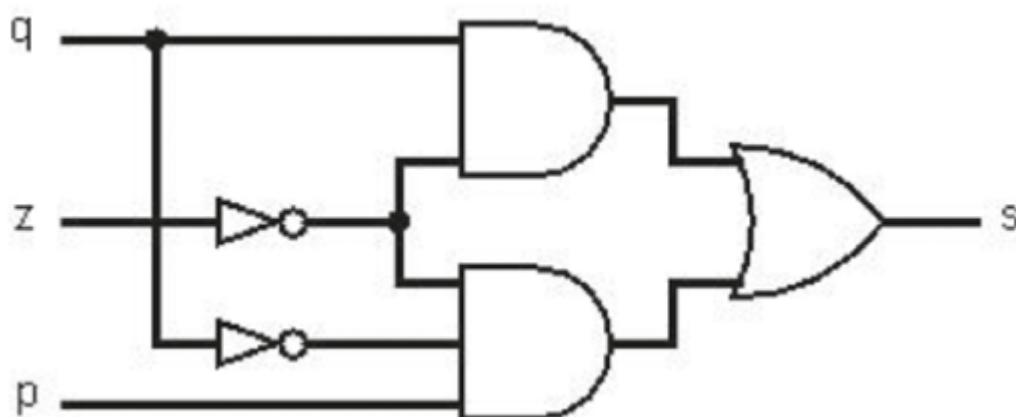


Figura 8 | Diagrama de porta automática. Fonte: Tangon e dos Santos (2016, p. 212).

Saiba mais

Saiba mais

Adquira mais conhecimento sobre os conteúdos desta aula, por meio das leituras a seguir:

- Fundamentos de portas lógicas:

DAGHLIAN, J. Portas Lógicas. *In*: DAGHLIAN, J. [Lógica e álgebra de Boole](#). 4. ed. São Paulo: Atlas, 2012. p. 154-165.

- Tipos de portas lógicas:

LOURENÇO, A. C. D. *et al.* Álgebra Booleana e Circuitos Combinacionais. *In*: LOURENÇO, A. C. D. *et al.* [Circuitos Digitais - Estude e Use](#). 4. ed. São Paulo: Érica, 2007. p. 46-139.

- Representação das portas lógicas:

DOS SANTOS, V. [Funções e portas lógicas](#). Computer Science Master, 2016.

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

Referências

Referências

FLOYD, T. **Sistemas digitais: Fundamentos e Aplicações**. 9. ed. Porto Alegre: Bookman, 2007.

STALLINGS, W. **Arquitetura e organização de computadores**. 10. ed. São Paulo: Pearson, 2017.

TANENBAUM, A. S. **Organização estruturada de computadores**. 5. ed. São Paulo, SP: Pearson, 2007.

TANGON, L.; DOS SANTOS, R. C. **Arquitetura e Organização de Computadores**. Londrina: Editora e Distribuidora Educacional S.A, 2016.

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

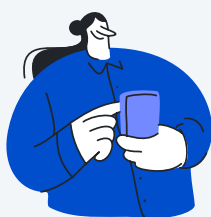
TOCCI, R. J.; WIDMER, N. S. **Sistemas digitais: princípios e aplicações**. 11. ed. São Paulo: Prentice-Hall, 2011.

Aula 4

Introdução a Circuitos

Introdução a circuitos

Introdução a circuitos



Este conteúdo é um vídeo!

Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Estudante, esta videoaula foi preparada especialmente para você. Nela, você irá aprender conteúdos importantes para a

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

sua formação profissional. Vamos assisti-la?

[Clique aqui](#) para acessar os slides da sua videoaula.

Bons estudos!

Ponto de Partida

Ponto de Partida

Convidamos você a embarcar em uma jornada exploratória pelo mundo dos circuitos. Esta aula abordará uma introdução aos circuitos, na qual você descobrirá a relação entre representações analógicas e digitais, e como a lógica combinacional e sequencial se unem-se para formar sistemas que são a base de todos os dispositivos eletrônicos que permeiam o nosso cotidiano.

Nossa exploração incluirá:

- Os princípios básicos de circuitos para computação.
- Lógica combinacional e lógica sequencial.
- Circuitos integrados.

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

Para solidificar o seu aprendizado, você será guiado por meio de uma atividade prática que o ajudará a visualizar conceitos abstratos e aplicá-los em situações do mundo real. Imagine que você é um engenheiro encarregado de projetar um multiplexador. Multiplexadores, dispositivos de circuito combinacional, consolidam múltiplas entradas em uma única saída. Equipados com variáveis seletoras, eles filtram e direcionam os dados de entrada para produzir um único resultado de saída. Entre as suas várias utilidades, a capacidade de criar funções lógicas específicas é uma aplicação destacada dos multiplexadores (TANGON; DOS SANTOS, 2016). Apresentaremos o seu diagrama esquemático com uma breve descrição do funcionamento do circuito para que você elabore a tabela verdade deste a partir de uma função. Ao final desta aula, você não apenas terá um entendimento robusto dos princípios que governam os circuitos elétricos, mas também estará equipado com o conhecimento para começar a criar os seus próprios projetos de circuitos. Bons estudos!

Vamos Começar!

Vamos Começar!

Princípios básicos de circuitos para computação

Na área de engenharia e em outras áreas profissionais, é essencial administrar de maneira ágil a abundância de dados e informações requisitados. Esses dados podem ser armazenados, monitorados, mensurados, visualizados e empregados conforme necessário, atendendo às demandas específicas do mercado relevante. Com eficiência e precisão, manipulamos dados convertidos em informações, que são processadas conforme o sistema computacional. Dessa forma, abordaremos a questão da representação e quantificação de um grande número de conjuntos de dados. Existem duas abordagens principais para o tratamento da comunicação de dados em bits entre máquinas, infraestruturas e aplicações, permitindo-nos quantificar e codificar esses dados em linguagem binária (TOCCI; WIDMER, 2011):

- **Representação analógica:** este formato utiliza dispositivos capazes de manipular quantidades físicas que variam continuamente em um determinado

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

intervalo. Um exemplo seria o ajuste de volume em um alto-falante, que pode oscilar entre o silêncio e o nível máximo de saída.

- **Representação digital:** sistemas digitais, por outro lado, operam com dispositivos projetados para gerenciar informações lógicas ou representações físicas em formato digital, em que os dados existem em valores discretos. A maioria desses dispositivos é na versão eletrônica, mas também podem ser encontrados em versões magnéticas, mecânicas ou pneumáticas.

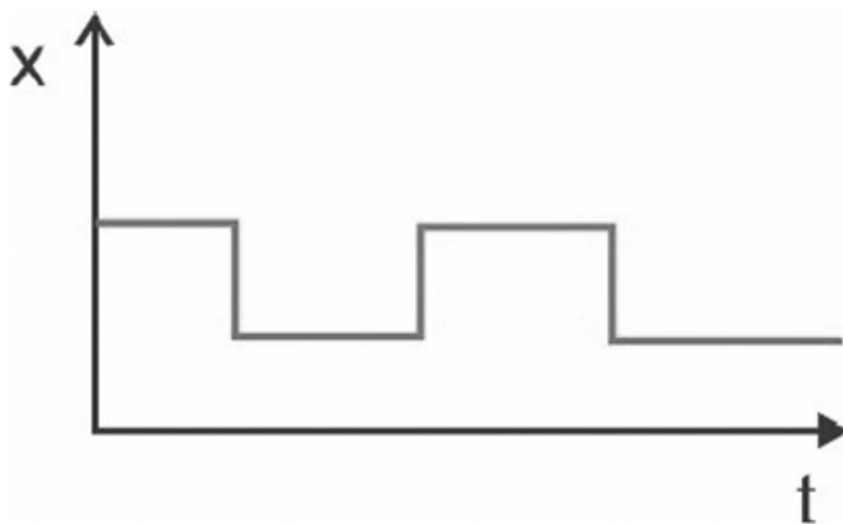


Figura 1 | Demonstração de valores em sistemas digitais.
Fonte: Tangon e dos Santos (2016, p. 218).

Siga em Frente...

Siga em Frente...

Lógica combinacional e lógica sequencial

Os sinais digitais são codificados como uma série de zeros e uns, que são transmitidos ao longo de um intervalo de tempo definido pelo relógio do sistema, notado como T . Ao lidar com a digitalização, enfrentamos duas questões primordiais: a predominância do mundo analógico em nosso ambiente e a temporalidade inerente ao processamento de sinais digitalizados. Para abordar esses desafios, seguimos um processo de quatro etapas:

1. A conversão de uma variável física em um sinal elétrico analógico.
2. A transformação do sinal analógico recebido em formato digital.
3. O processamento da informação digital por meio de operações computacionais.
4. A reconversão do resultado digital em um sinal analógico para saída.

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

Além disso, a eletrônica digital é categorizada em dois ramos principais: a lógica combinacional, na qual a saída é uma função direta das entradas atuais, e a lógica sequencial, que depende tanto das entradas atuais quanto do estado anterior do sistema (TANGON; DOS SANTOS, 2016).

A lógica combinacional é um conceito fundamental na eletrônica digital que se refere a circuitos em que as saídas são uma combinação atual das entradas. Diferentemente da lógica sequencial, em que o histórico das entradas afeta a saída, na lógica combinacional, a saída é determinada exclusivamente pelas entradas presentes, independentemente das condições anteriores (LOURENÇO *et al.*, 2007).

O sinal "*Enable*"

Dentro dos circuitos combinacionais, um conceito importante é o do sinal "*Enable*" (En), frequentemente encontrado em diversos dispositivos, como multiplexadores, decodificadores e registros. O sinal "*Enable*" atua como uma chave que controla se um determinado componente do circuito está ativo ou inativo. Quando o sinal "*Enable*" está em estado lógico alto (1), o circuito está habilitado para operar ou transmitir dados. Por outro lado, quando o sinal

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

está em estado lógico baixo (0), o circuito fica inoperante ou desabilitado, e não permite a passagem de dados.

Considere um circuito simples com uma porta *AND* de uma entrada A e um sinal "En". A saída será representada por S.

EN	ENTRADA (A)	SAÍDA (S)
0	0	0
0	1	0
1	0	0
1	1	1

Tabela 1 | Sinal En. Fonte: elaborada pela autora.

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

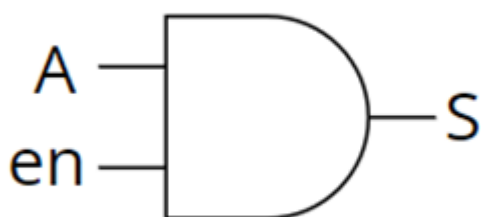


Figura 2 | Diagrama En. Fonte: elaborada pela autora.

Segue uma lista revisada de circuitos combinacionais fundamentais:

- **Circuito de habilitação/deshabilitação:** controla o estado ativo de outros circuitos ou componentes.
- **Codificador:** converte informações de várias linhas em um código mais simples.
- **Multiplexador:** seleciona uma entrada dentre várias e direciona para uma única saída.
- **Decodificador:** realiza a operação inversa do codificador, convertendo códigos de entrada em um conjunto específico de saídas.
- **Demultiplexador:** distribui um sinal de entrada para várias saídas, sendo o inverso do multiplexador.

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

- **Gerador de paridade:** produz um bit de paridade com base na quantidade de bits '1' em uma palavra.
- **Verificador de paridade:** checa a integridade de dados utilizando o bit de paridade.
- **Comparador:** determina a igualdade ou relação de grandeza entre duas entradas binárias.
- **Circuitos aritméticos:** incluem componentes, como somadores e subtratores, para realizar operações aritméticas.
- **Somador:** calcula a soma de duas entradas binárias.
- **Shifter (Deslocador):** altera a posição dos bits em uma palavra, para a esquerda ou direita.
- **Subtratores:** efetuam a subtração de números binários.

A lógica sequencial difere da lógica combinacional por ter saídas que dependem não só do estado atual de suas entradas, mas também do histórico destas. Ou seja, a lógica sequencial leva em conta a sequência de eventos ao longo do tempo, o que é fundamental para a criação de memórias e sistemas de computação que necessitam manter um estado ou "lembrar" informações passadas (LOURENÇO *et al.*, 2007).

Os circuitos sequenciais são tipicamente compostos por portas lógicas e elementos de armazenamento temporário,

como *flip-flops* e *latches*, que são capazes de armazenar um bit de informação. A função desses elementos é manter um estado até que um sinal de controle, frequentemente um pulso de *clock*, sinalize uma atualização.

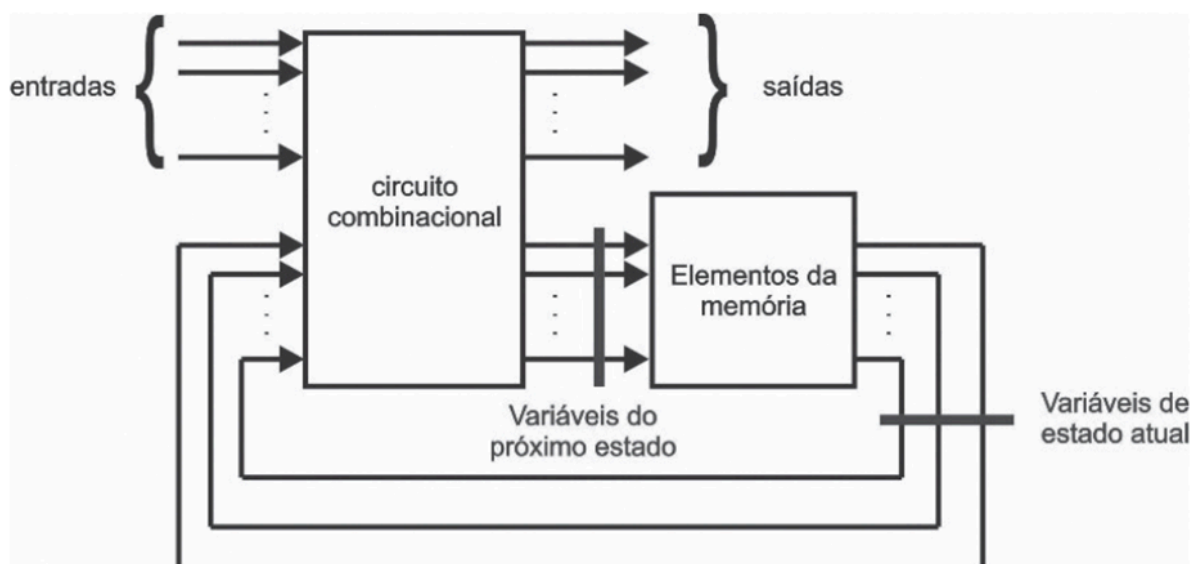


Figura 3 | Diagrama de blocos de um circuito sequencial.
Fonte: Tangon e dos Santos (2016, p. 222).

O *flip-flop*

Um exemplo clássico de um circuito de lógica sequencial é o *flip-flop*, que é a base para a construção de memória e contadores em eletrônica digital. Ele pode manter um estado binário até que seja instruído a mudar por um sinal de *clock*.

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

Vamos considerar um *flip-flop* tipo D simples, que possui uma entrada de dados (D), uma entrada de *clock* (CLK), e uma saída (Q). O estado da saída Q segue o estado da entrada D sempre que ocorre uma borda ascendente no sinal de *clock*. Isso significa que se D está alto (1) quando o *clock* pulsa, Q se tornará alto. Se D está baixo (0) no pulso do *clock*, Q se tornará baixo.

CLK (Clock)	D (Entrada de Dados)	Q (Saída Anterior)	Q (Saída Atual)
Bordas	X	Qn	Qn
Borda	0	Qn	0
Borda	1	Qn	1

Tabela 2 | Tabela verdade para um *flip-flop* tipo D. Fonte: elaborada pela autora.

Na Tabela 2, "Bordas / " representa as bordas descendentes ou ascendentes do *clock*, nas quais não há mudança, "X" indica um estado irrelevante, e "Qn" é o estado anterior da saída. A saída muda apenas com a borda ascendente do sinal de *clock*.

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

O *latch*

Os *latches* representam a estrutura mais elementar para a construção de um circuito de memória básico. O termo, que em português se refere a fechaduras ou trincos, reflete a função de retenção desses dispositivos. Constituídos por duas portas lógicas que operam em negação recíproca, eles apresentam duas saídas principais: uma variável lógica e o seu inverso. Conforme descrevem Tangon e Dos Santos (2016), a configuração de um *latch* permite que o sinal de saída seja reintroduzido na entrada, criando um ciclo de realimentação. Essa configuração de realimentação é o que permite que o *latch* mantenha um dos dois possíveis estados estáveis, com as saídas conectadas de forma cruzada às entradas, garantindo a manutenção do estado até que um novo sinal seja recebido (LOURENÇO *et al.*, 2007).

Aplicações de lógica sequencial

Os circuitos sequenciais são usados para criar máquinas de estado, contadores, registro etc. Eles são essenciais para o desenvolvimento de computadores, enquanto gerenciam desde a execução de instruções em um processador até o armazenamento de dados em memórias voláteis e não voláteis.

Circuitos integrados

Circuitos integrados (CIs), também conhecidos como chips, representam um dos maiores avanços na eletrônica. São dispositivos microeletrônicos em que componentes, como resistores, capacitores, transistores e outros, são fabricados em uma pequena placa de material semicondutor, geralmente silício. Jack Kilby e Robert Noyce são creditados com a invenção do circuito integrado na década de 1950, momento que deu início à era da microeletrônica (MONTEIRO, 2010).

Características e tipos de circuitos integrados

Circuitos integrados são classificados pelo número de componentes que contêm:

- **SSI (*Small-Scale Integration*)**: contêm dezenas de componentes por chip.
- **MSI (*Medium-Scale Integration*)**: apresentam centenas de componentes.
- **LSI (*Large-Scale Integration*)**: incorporam milhares de componentes.

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

- **VLSI (*Very-Large-Scale Integration*)**: têm dezenas de milhares a milhões de componentes.

Exemplos de circuitos integrados

- **Microprocessadores**: compõe o cérebro de computadores e dispositivos eletrônicos. Um microprocessador típico, como o Intel Core i7, é um exemplo de um CI VLSI que contém bilhões de transistores e executa uma variedade de tarefas computacionais complexas.
- **Memória**: chips de memória, como o DRAM (*Dynamic Random-Access Memory*), são CIs utilizados para armazenar dados temporariamente em computadores e smartphones. Um módulo de memória DDR4 é um exemplo que pode conter milhões de células de memória para armazenar informações.
- **Amplificadores operacionais**: amplificadores operacionais (*op-amps*) são CIs usados para amplificar sinais elétricos e são encontrados em dispositivos de áudio e instrumentação. O LM741 é um exemplo clássico de um CI de amplificador operacional.
- **Reguladores de tensão**: são CIs utilizados para manter uma tensão constante em um circuito. O LM7805 é um

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

regulador de tensão que fornece uma saída de 5V a partir de uma entrada de tensão mais alta.

Circuitos integrados são a base da eletrônica moderna e da computação. Desde o seu advento, eles têm possibilitado o desenvolvimento de tecnologias avançadas, desde telefones celulares a satélites. A compreensão dos CIs e de como eles são projetados e aplicados é essencial para estudantes e profissionais da área de eletrônica e ciências da computação, pois eles continuam a ser a força motriz por trás da inovação tecnológica.

Vamos Exercitar?

Vamos Exercitar?

No “Ponto de partida” desta aula, apresentamos a você uma situação-problema em que o seu objetivo era projetar um circuito do tipo multiplexador.

Antes de começarmos a solucionar, é essencial compreender a definição de um mutiplexador, já apresentada

anteriormente: multiplexadores, dispositivos de circuito combinacional, consolidam múltiplas entradas em uma única saída. Equipados com variáveis seletoras, eles filtram e direcionam os dados de entrada para produzir um único resultado de saída. Entre as suas várias utilidades, a capacidade de criar funções lógicas específicas é uma aplicação destacada dos multiplexadores (TANGON; DOS SANTOS, 2016). A função utilizada para esse multiplexador é:

$$Z = A\overline{B}\overline{C} + \overline{A}B\overline{C} + ABC$$

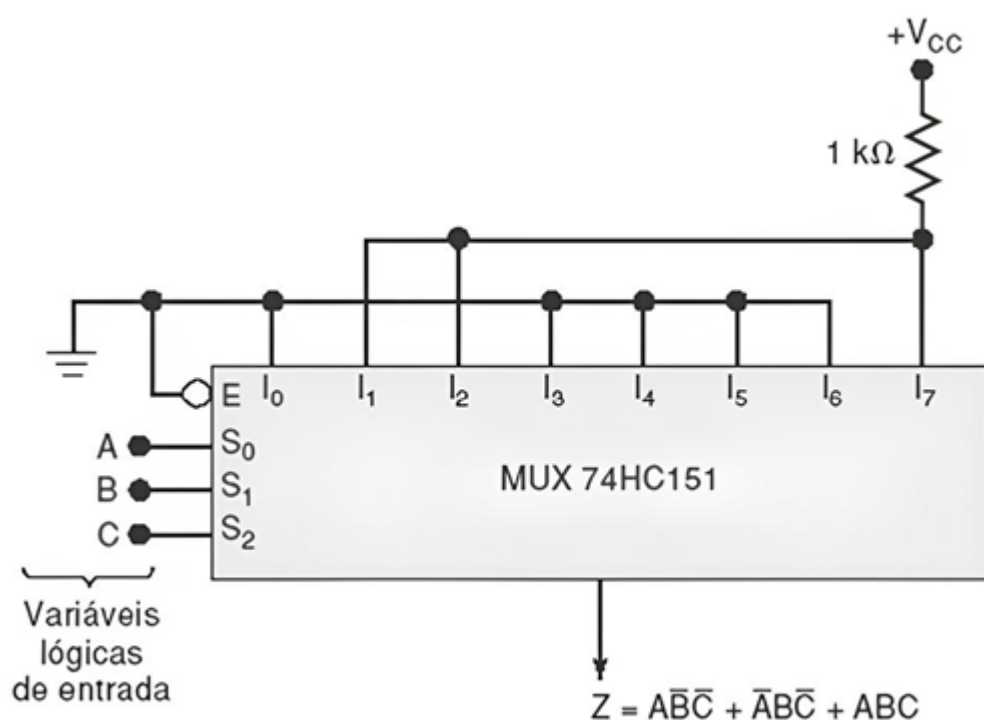


Figura 4 | Multiplexador com três variáveis de entrada.
Fonte: Tangon e Santos (2016, p. 226).

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

Para montarmos a tabela verdade solicitada, devemos montá-la com as entradas A, B e C e a saída Z, conforme a Tabela 3.

ENTRADAS			SAÍDA
A	B	C	Z
0	0	0	0
1	0	0	1
0	1	0	1
1	1	0	A
0	0	1	0
1	0	1	0
0	1	1	0
1	1	1	1

Tabela 3 | Tabela verdade resultante. Fonte: Tangon e dos Santos (2016, p. 226).

Explicação da tabela verdade:

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

- Desse modo, temos os valores de Z, se substituirmos os valores de A, B e C na expressão:

$$Z = A\overline{B}\overline{C} + \overline{A}B\overline{C} + ABC$$

Saiba mais

Saiba mais

Aprofunde seus conhecimentos sobre os conteúdos desta aula com as indicações a seguir:

- Os princípios básicos de circuitos para computação:

NETO, A.; OLIVEIRA, Y. D. Introdução à eletrônica digital. *In*: NETO, A.; OLIVEIRA, Y. D. [Eletrônica analógica e digital aplicada à IoT: aprenda de maneira descomplicada](#). Rio de Janeiro: Alta Books, 2020. p. 144-198.

- Circuitos integrados

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

ALVES, P. [O que é circuito integrado? Tipos e aplicações!](#)
Manual da eletrônica, [s. d.].

Referências

Referências

LOURENÇO, A. C. D. *et al.* **Circuitos Digitais – Estude e Use.** 9. ed. São Paulo: Érica, 2007.

MONTEIRO, M. A. **Introdução à Organização de Computadores.** 5. ed. Rio de Janeiro: LTC, 2010.

PATTERSON, D.; HENNESSY, J. **Organização e Projeto de Computadores.** 5. ed. Rio de Janeiro: Elsevier, 2017.

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

STALLINGS, W. *Arquitetura e organização de computadores*. 10. ed. São Paulo: Pearson, 2017.

TANGON, L.; DOS SANTOS, R. C. *Arquitetura e Organização de Computadores*. Londrina: Editora e Distribuidora Educacional S.A, 2016.

TOCCI, R. J.; WIDMER, N. S. *Sistemas digitais: princípios e aplicações*. 11. ed. São Paulo: Prentice-Hall, 2011.

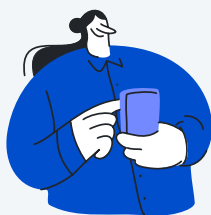
Aula 5

Encerramento da Unidade

Videoaula de Encerramento

Videoaula de Encerramento

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES



Este conteúdo é um vídeo!

Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Estudante, esta videoaula foi preparada especialmente para você. Nela, você irá aprender conteúdos importantes para a sua formação profissional. Vamos assisti-la?

[Clique aqui](#) para acessar os slides da sua videoaula.

Bons estudos!

Ponto de Chegada

Ponto de Chegada

Olá, estudante! Para desenvolver a competência desta unidade, que é integrar o conhecimento teórico da álgebra

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

booleana com a prática da lógica digital para criar soluções eficientes de hardware que atendam a requisitos específicos de desempenho, eficiência e funcionalidade na construção de sistemas computacionais modernos, é preciso conhecer os conceitos fundamentais que formam a base da computação digital.

Na aula **“Introdução à álgebra booleana”**, exploramos as operações e as leis básicas que regem o sistema binário, formando o alicerce para todas as funções lógicas que você encontrará em circuitos digitais.

Avançando para a aula **“Expressões lógicas”**, aplicamos o conhecimento teórico da álgebra booleana para formular e simplificar expressões lógicas. Essa habilidade é essencial para otimizar o design de circuitos digitais, permitindo que você crie soluções mais eficientes.

A aula **“Portas lógicas”** introduziu os componentes básicos dos circuitos digitais. Você aprendeu a identificar e aplicar as diversas portas lógicas, como *AND*, *OR*, *NOT*, *NAND*, *NOR*, *XOR* e *XNOR*, que são usadas para processar dados binários.

Por fim, na aula **“Introdução a circuitos”**, você começou a juntar todos os conceitos anteriores para entender como os

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

circuitos digitais são construídos e como eles formam os sistemas computacionais complexos que usamos em nossa vida cotidiana.

Ao compreender como a álgebra booleana é aplicada no design de circuitos e sistemas digitais, você está bem equipado para enfrentar desafios de engenharia e desenvolvimento tecnológico, podendo traduzir teoria em prática efetiva.

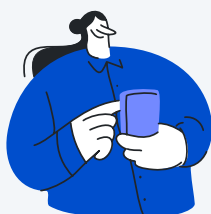
Refleta

Ponderando sobre os conteúdos desta unidade, é importante que você reflita sobre as seguintes questões para aprimorar o seu entendimento e a aplicação prática:

1. Como a simplificação de uma expressão lógica pode influenciar a eficiência de um circuito digital?
2. De que maneira as portas lógicas podem ser combinadas para realizar uma função de memória em um circuito?
3. Quais são as considerações mais importantes ao transitar do design teórico de um circuito para a sua implementação prática?

É Hora de Praticar!

É Hora de Praticar!



Este conteúdo é um vídeo!

Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Sistema de controle de acesso

Você está projetando um sistema de controle de acesso para uma sala de segurança que tem duas fechaduras eletrônicas (A e B) e um sensor de movimento (S). As regras para o sistema permitir o acesso (sinal de saída P) são as seguintes:

- ambas as fechaduras, A e B, devem estar desbloqueadas (1) simultaneamente; ou
- o sensor de movimento deve detectar movimento (1) quando pelo menos uma das fechaduras estiver desbloqueada (1).

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

As condições acima devem ser traduzidas em uma expressão lógica que determinará se o acesso é permitido ($P = 1$) ou negado ($P = 0$).

Elabore a expressão lógica, identifique quais portas lógicas são sugeridas e o seu diagrama de circuitos.

Refleta

Como você pretende solucionar o desafio proposto? Quais aprendizados dessa prática você pode levar para a sua futura realidade profissional?

Resolução do Estudo de Caso

O circuito proposto leva em conta as duas condições para permitir acesso. A primeira condição é direta: ambas as fechaduras devem estar desbloqueadas, o que é uma operação lógica E (*AND*). A segunda condição é mais complexa, pois o movimento deve ser detectado apenas quando pelo menos uma fechadura está desbloqueada, o que requer uma operação lógica OU (*OR*) entre as fechaduras seguida de uma operação E (*AND*) entre o resultado e o sensor de movimento.

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

Expressão lógica: $P = (A \text{ AND } B) \text{ OR } (S \text{ AND } (A \text{ OR } B))$

O uso de portas *OR* e *AND* aqui é coerente com a álgebra booleana que define as operações lógicas de conjunção (*AND*) e disjunção (*OR*). O projeto final do circuito deve refletir a expressão lógica criada e garantir que todas as condições para o acesso sejam satisfeitas para que o sinal de permissão seja ativado ($P = 1$).

Portas lógicas sugeridas:

- Para a expressão $(A \text{ AND } B)$, utilizaremos uma porta lógica *AND*.
- Para a expressão $(A \text{ OR } B)$, uma porta lógica *OR*.
- Para a expressão final, outra porta lógica *OR* que combina as saídas das duas expressões anteriores.

Por fim, para o **diagrama de circuitos**, utilize:

- Uma porta *AND* recebendo as entradas A e B.
- Uma porta *OR* recebendo as entradas A e B.
- Uma porta *AND* recebendo a entrada do sensor de movimento S e a saída da porta *OR* acima.
- Uma porta *OR* final que combina as saídas das duas portas *AND* para produzir o resultado P.

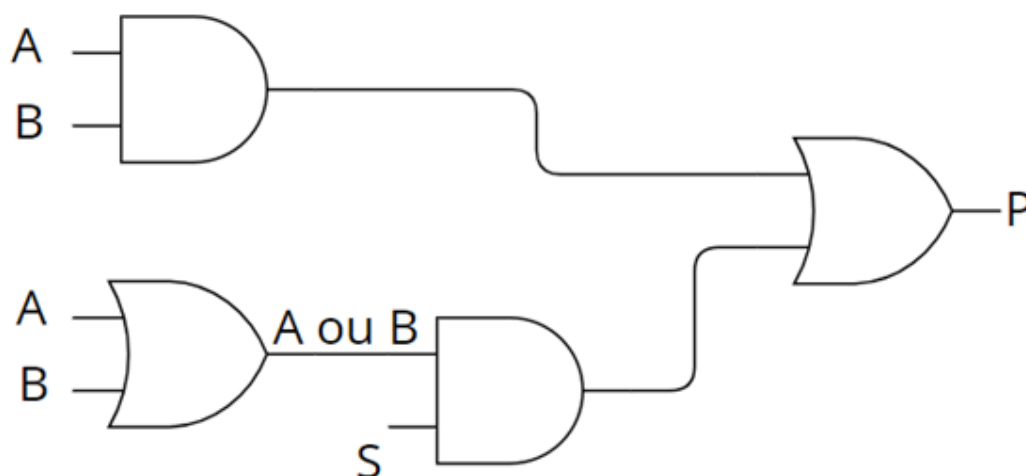


Figura 1 | Digrama da solução. Fonte: elaborada pela autora.

Esse problema ilustra como a álgebra booleana é usada para criar expressões lógicas que podem ser implementadas fisicamente com portas lógicas em circuitos, formando a base dos sistemas digitais.

Assimile

Por meio do infográfico a seguir, vamos entender como a álgebra booleana permite que os dispositivos eletrônicos processem informações e realizem ações complexas,

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

sustentando tudo, desde smartphones até computadores avançados.

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES



Fonte: elaborada pela autora.

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

Referências

FLOYD, T. **Sistemas digitais: Fundamentos e Aplicações**. 9. ed. Porto Alegre: Bookman, 2007.

STALLINGS, W. **Arquitetura e organização de computadores**. 10. ed. São Paulo: Pearson, 2017.

TANENBAUM, A. S. **Organização estruturada de computadores**. 5. ed. São Paulo, SP: Pearson, 2007.

TANGON, L.; DOS SANTOS, R. C. **Arquitetura e Organização de Computadores**. Londrina: Editora e Distribuidora Educacional S.A, 2016.

TOCCI, R. J.; WIDMER, N. S. **Sistemas digitais: princípios e aplicações**. 11. ed. São Paulo: Prentice-Hall, 2011.